

Escuela de Ingeniería Informática
Universidad de Las Palmas de Gran Canaria

Proyecto

Sistema de salas para jugar al ajedrez

Arquitectura de sistemas, frontend, backend y arquitectura hexagonal

Índice

1. Objetivos	1
2. Requisitos	1
3. Entregables	3
3.1. Organización del repositorio	3
3.2. Contenido del README	4
4. Evaluación	5

1 Objetivos

Tipo de entrega

Esta entrega es un proyecto individual, no una kata. No debe repetirse ni lleva vídeo explicativo. Se entrega en un repositorio público. El README es la explicación evaluable: debe incluir ejecución de frontend y backend, arquitectura de sistemas, arquitectura hexagonal, capturas obligatorias, verificación e ideas propias.

Este proyecto plantea el desarrollo de un sistema para crear salas de ajedrez y jugar partidas entre usuarios con una separación clara entre frontend y backend. No es una kata: no se repite varias veces ni lleva vídeo explicativo.

La entrega debe permitir crear una sala, unirse a una sala existente, jugar o reproducir una partida desde el frontend, comunicar ese frontend con un backend y registrar la información esencial de las partidas para poder consultarla después.

El trabajo debe diseñarse como un sistema con dos aplicaciones coordinadas y contratos explícitos entre ellas. Tanto frontend como backend deben aplicar arquitectura hexagonal de forma proporcionada al alcance: dominio y casos de uso en el centro, puertos definidos por la aplicación y adaptadores para UI, HTTP, persistencia o comunicación.

Se anima al alumno a incorporar ideas propias dentro del alcance del proyecto: por ejemplo, salas privadas con código, invitaciones, lista de salas disponibles, chat de sala, modo espectador, historial de partidas, reloj, análisis básico de movimientos o pequeñas mejoras de experiencia.

Competencias trabajadas

- Diseñar una arquitectura de sistemas con frontend, backend y contratos de comunicación.
- Diseñar un frontend con tablero de ajedrez, creación de salas e interacción básica.
- Diseñar un backend con API o comunicación bidireccional.
- Aplicar arquitectura hexagonal en backend: casos de uso, puertos de entrada, puertos de salida y adaptadores.
- Aplicar arquitectura hexagonal en frontend de forma ligera: interfaz, casos de uso de interacción, puerto de servicio y adaptador de comunicación.
- Modelar usuarios, partidas, movimientos y estados de partida.
- Registrar partidas de forma persistente.
- Poner en contacto a usuarios mediante salas, invitaciones, emparejamiento o códigos de partida.
- Separar interfaz, comunicación, dominio, persistencia y configuración.
- Documentar ejecución local de frontend y backend.

2 Requisitos

Enunciado

Desarrolla un sistema de salas de ajedrez formado por al menos dos partes: un frontend y un backend. El frontend debe permitir crear o entrar en una sala y ofrecer un tablero sobre el que se puedan realizar movimientos o reproducir una partida. El backend debe coordinar salas, usuarios o sesiones, registrar partidas y ofrecer algún mecanismo para poner en contacto a dos usuarios.

La idea base es que un usuario pueda crear una sala y otro pueda unirse para compartir una partida. También se aceptan variantes como invitaciones, emparejamiento sencillo, lista de salas disponibles, modo espectador, chat de sala o reproducción de partidas registradas.

No es obligatorio implementar un motor de ajedrez completo ni una inteligencia artificial. Sí debe quedar claro qué reglas se validan, qué movimientos se registran y qué responsabilidades pertenecen al frontend y al backend.

La arquitectura debe explicarse en dos niveles: arquitectura de sistemas, mostrando frontend, backend, base de datos y contratos; y arquitectura interna, mostrando cómo se aplican puertos y adaptadores dentro de cada aplicación.

Desarrollo de la entrega e ideas propias

Esta entrega se desarrolla una sola vez como proyecto. Se espera una construcción progresiva, con commits que muestren avance real, y una aportación personal del alumno: una pequeña mejora funcional, decisión de arquitectura, detalle de comunicación o mejora de experiencia que pueda justificarse en el README sin desviar el objetivo principal.

- Diseña primero el modelo mínimo: usuario, partida, movimiento, estado de partida y resultado si procede.
- Define los contratos entre frontend y backend: rutas HTTP para salas y partidas, eventos WebSocket, mensajes de socket u otra alternativa documentada.
- Implementa un backend hexagonal que permita crear salas, unirse a salas, registrar movimientos y consultar el histórico.
- Implementa un frontend con separación entre UI del tablero, casos de uso de interacción y adaptador de comunicación con el backend.
- Añade un mecanismo para conectar usuarios: sala compartida, invitación, emparejamiento simple, código de sala o lista de salas disponibles.
- Incorpora una idea propia ajustada al alcance del proyecto y descríbela en el README.
- Cierra la entrega con persistencia, tratamiento de errores, capturas obligatorias y verificación reproducible.

Requisitos funcionales

1. Frontend con tablero de ajedrez visible, creación o selección de sala e interacción mínima.
2. Backend ejecutable de forma independiente.
3. Comunicación frontend/backend mediante HTTP, WebSocket, sockets u otra alternativa documentada.
4. Contrato de comunicación documentado: rutas, eventos, mensajes, cuerpos JSON o protocolo usado.
5. Backend organizado con arquitectura hexagonal: dominio, casos de uso, puertos y adaptadores.
6. Frontend organizado con separación hexagonal ligera: UI como adaptador de entrada, puerto de servicio para comunicarse con backend y adaptador HTTP/WebSocket/socket.
7. Registro persistente de partidas con identificador de sala o partida, jugadores, movimientos y fecha o estado.
8. Mecanismo para poner usuarios en contacto: sala, invitación, emparejamiento o código compartido.
9. Consulta del estado de una partida o de su histórico.
10. Gestión de errores: partida inexistente, movimiento inválido según las reglas implementadas, usuario desconectado o datos incompletos.
11. Configuración documentada de puertos, base de datos y ejecución.

Criterios de diseño

- El frontend no debe escribir directamente en la base de datos.
- El backend no debe depender de componentes visuales del frontend.
- El modelo de partida debe estar separado de la capa de transporte.
- La persistencia debe quedar encapsulada en un adaptador de salida que implemente un puerto definido por la aplicación.
- Los casos de uso del backend no deben depender del framework HTTP, WebSocket o socket elegido.
- La UI del frontend no debe construir directamente peticiones o mensajes de red si puede delegar en un puerto de servicio.
- El adaptador de comunicación del frontend debe ser sustituible por un mock o doble de prueba.
- La validación de movimientos debe estar documentada: reglas completas, subconjunto implementado o límites explícitos.
- No subas bases de datos con datos personales ni credenciales al repositorio.
- Evita rutas absolutas y configuración fija de máquina local.

Verificación esperada

- Ejecuta backend y frontend desde una copia limpia del repositorio.
- Crea una sala, une dos usuarios y registra varios movimientos.
- Conecta dos frontends o simula dos usuarios sobre la misma sala.
- Consulta el histórico de una partida registrada.
- Prueba un movimiento o petición inválida y documenta la respuesta.
- Sustituye al menos un adaptador por un doble de prueba o demuestra cómo podría hacerse sin tocar el caso de uso.
- Reinicia el backend y comprueba si la persistencia conserva las partidas si esa es la opción implementada.

3 Entregables

3.1 Organización del repositorio

El repositorio de esta entrega debe llamarse chess-software. Su ubicación esperada es [https://github.com/\[tu cuenta\]/chess-software](https://github.com/[tu cuenta]/chess-software).

El repositorio debe ser público.

- La rama principal debe contener la versión final evaluable.
- Los commits deben mostrar avance progresivo: creación, implementación, verificación, refactorización y documentación.
- No deben subirse binarios ni resultados de compilación: target, out, build, bin, .class, .jar, .war, .zip, .exe ni artefactos temporales.
- El repositorio debe incluir un .gitignore adecuado para Java e IntelliJ IDEA.

- Deben incluirse los ficheros necesarios para reproducir la entrega: pom.xml o build.gradle, src, README.md y recursos pequeños imprescindibles.
- Nunca deben subirse credenciales, tokens, claves de API ni rutas personales.

3.2 Contenido del README

- Objetivo de la entrega.
- Cómo compilar y ejecutar.
- Dependencias y versión de JDK.
- Estructura de la entrega y clases principales.
- Arquitectura de sistemas con frontend, backend, base de datos y contrato de comunicación.
- Arquitectura hexagonal del backend con clases, paquetes o módulos concretos.
- Arquitectura hexagonal ligera del frontend con clases, paquetes o módulos concretos.
- Cómo ejecutar backend y frontend, incluyendo puertos, base de datos y dependencias.
- Modelo de datos usado para usuarios, salas, partidas y movimientos.
- Mecanismo para conectar usuarios: sala, invitación, emparejamiento o código compartido.
- Idea propia incorporada y justificación de por qué encaja con el trabajo.
- Capturas obligatorias del tablero funcionando, de una sala y de una partida o histórico registrado.
- Verificación realizada: pruebas automatizadas o comprobaciones manuales documentadas.

Aspectos que deben quedar claros en el README

- Arquitectura de sistemas con diagrama simple: frontend, backend, base de datos y contratos.
- Arquitectura hexagonal del backend: dominio, casos de uso, puertos y adaptadores.
- Arquitectura hexagonal ligera del frontend: UI, casos de uso, puerto de servicio y adaptador de comunicación.
- Cómo ejecutar backend y frontend, incluyendo puertos y dependencias.
- Contrato de comunicación entre frontend y backend.
- Modelo de datos: usuarios, salas, partidas y movimientos.
- Mecanismo elegido para crear salas y conectar usuarios.
- Capturas obligatorias del tablero, de la creación o unión a una sala y de una partida o histórico registrado.
- Idea propia incorporada y motivo por el que mejora el trabajo.
- Límites conocidos: reglas de ajedrez implementadas, persistencia, concurrencia o comunicación.

Elementos finales a entregar

- Repositorio chess-software.
- README con ejecución de frontend y backend, arquitectura de sistemas, arquitectura hexagonal y configuración.
- Explicación en el README de las ideas propias incorporadas.

- Capturas obligatorias del tablero, de una sala y de una partida o histórico registrado.
- Evidencia de verificación de comunicación frontend/backend y registro de partidas.

4 Evaluación

Rúbrica específica

Criterio	Peso	Evidencias esperadas
Cumplimiento funcional	25 %	La solución permite usar un tablero, conectar usuarios y registrar partidas con flujos principales completos.
Arquitectura de sistema y hexagonal	25 %	Frontend, backend, contratos, puertos, casos de uso y adaptadores están separados con responsabilidades reconocibles.
Persistencia y trazabilidad	15 %	Las partidas, usuarios y movimientos quedan registrados de forma consultable y reproducible.
Aportación propia	15 %	La entrega incorpora una idea propia justificada, visible en el software y coherente con el objetivo del trabajo.
README	15 %	El README permite instalar, ejecutar, entender la arquitectura, ver capturas obligatorias y probar el sistema.
Verificación	5 %	Hay pruebas automatizadas o comprobaciones manuales documentadas de frontend, backend y casos de error.

Checklist antes de entregar

- La entrega compila desde cero.
- El README explica objetivo, ejecución y dependencias.
- El repositorio no contiene rutas absolutas personales ni credenciales.
- Los commits permiten entender el avance del proyecto.
- Frontend y backend se ejecutan y comunican de forma reproducible.
- El README incluye capturas obligatorias del tablero y del registro o histórico de partidas.
- El mecanismo para conectar usuarios está explicado y probado.
- La solución mantiene separadas interfaz, comunicación, dominio y persistencia.